



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet 2

JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P00 : **Java avancé**



Chapitre

3

Entrées/Sorties en Java

3- Entrées/Sorties

3.1 Introduction



- En informatique, le terme Entrées/Sorties (input/output) — abrégé E/S (ou I/O) désigne les **échanges de données** qui ont lieu **entre un programme** et le **monde extérieur**. Dans la plupart des cas, les données sont échangées entre le programme et :
 - l'utilisateur — via la console — et/ou
 - un ou plusieurs **fichiers**, et/ou
 - une ou plusieurs **connexions réseau (sockets)**.
- Ce chapitre se concentre essentiellement sur les entrées/sorties via des fichiers mais la plupart des notions s'appliquent également aux entrées/sorties via la console ou le réseau.

3- Entrées/Sorties

3.2 Fichier et système de fichiers



- Un fichier (**file**) est une **séquence d'octets** (bytes) stockée sur un support physique (tel que un disque dur, une clé USB, une carte mémoire SD, etc.).
- Le contenu d'un fichier est accompagné de méta-données (**metadata**) comportant des informations sur le fichier : son nom, ses droits d'accès, la date et l'heure de la dernière modification, etc.
- Un système de fichiers (**file system**) permet d'organiser un ensemble de fichiers dans des répertoires qui peuvent être imbriqués les uns dans les autres sous forme d'une structure arborescente.
 - Dans un tel système, un fichier ou répertoire est désigné par son chemin d'accès (**path**), composé de la liste des répertoires, dans lesquels il est imbriqué, et de son nom.
 - Dans la plupart des systèmes d'exploitation, les chemins d'accès sont représentés par des chaînes de caractères dans lesquelles

3- Entrées/Sorties

3.2 Fichier et système de fichiers



les noms des différentes composantes du chemin (répertoires et fichier) apparaissent dans l'ordre d'imbrication, séparées par un caractère de séparation (la barre oblique (/) pour les systèmes Unix (Linux, macOS, etc.) et la barre oblique inversée (\) pour Windows).

● Conventionnellement, on distingue **deux types de fichiers** :

- les fichiers **textuels** qui contiennent une séquence de caractères.
- les fichiers **binares** qui sont tous les autres fichiers : images, sons, vidéos, etc.





3- Entrées/Sorties

3.3 Entrées/Sorties en Java

● **L'API Java possède de nombreuses classes dédiées aux entrées/sorties qui sont réparties entre deux paquetages :**

- **java.io** contient les classes d'entrées/sorties d'origine
- **java.nio** (pour new io) contient des classes plus récentes (version 1 depuis Java 1.4, version 2 depuis Java 7).
 - Les classes de java.nio ne remplacent pas celles de java.io. Les deux paquetages sont nécessaires dans différentes situations, et sont souvent utilisées conjointement.

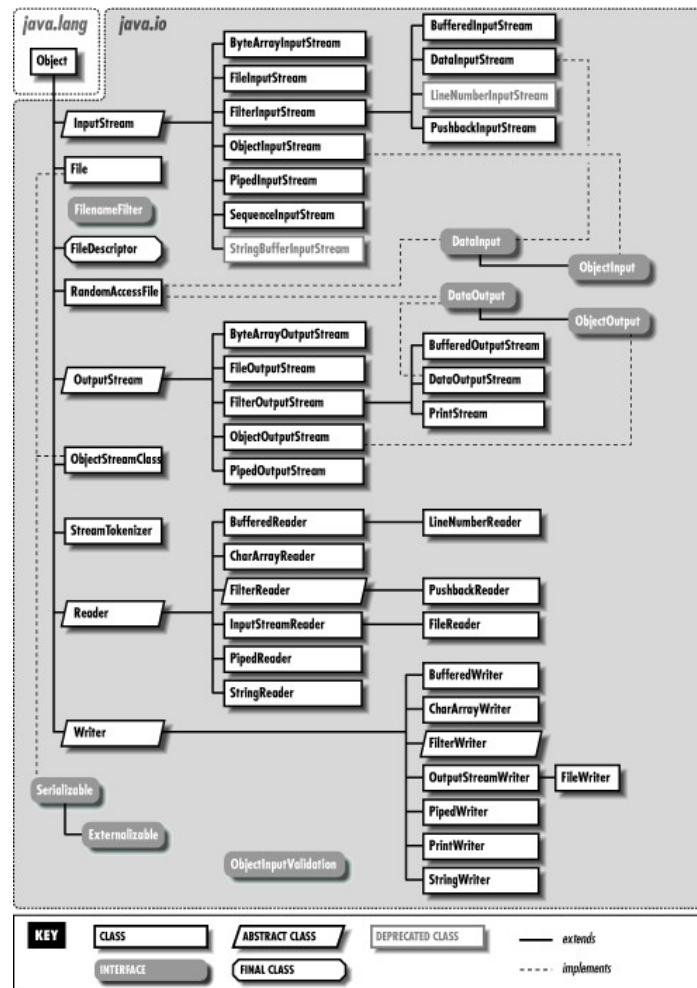
● **Le paquetage java.nio permet d'offrir une abstraction de base différente — et plus efficace — que celle de java.io. Cependant, java.io est plus simple à utiliser, et ses performances sont souvent assez bonnes.**

- L'abstraction de base de **java.io** est le flux (**stream**), tandis que celle de **java.nio** est la mémoire tampon (**buffer**).



3- Entrées/Sorties

3.3 Entrées/Sorties en Java



Package
java.io



3- Entrées/Sorties

3.3.1 Flux d'entrée/sortie



- **L'abstraction de base des E/S de java.io est le flux (appelé aussi stream ou flot).**
 - Un flux est une séquence de valeurs — qui peuvent être des octets ou des caractères — auxquelles on accède de manière séquentielle, c-à-d de la première à la dernière.
- **Un flux d'entrée est un flux dont les valeurs proviennent d'une source donnée — fichier, console ou connexion réseau — et sont lues par le programme.**
- **Un flux de sortie est un flux dont les valeurs sont fournies par le programme et écrites dans un fichier (console ou connexion réseau) qui les absorbe.**
- **L'API Java distingue deux types de flux :**
 - **Flux d'octets : input streams** pour les flux d'entrée d'octets, **output streams** pour ceux de sortie.
 - **Flux de caractères : readers** pour les flux d'entrée de caractères, **writers** pour ceux de sortie.

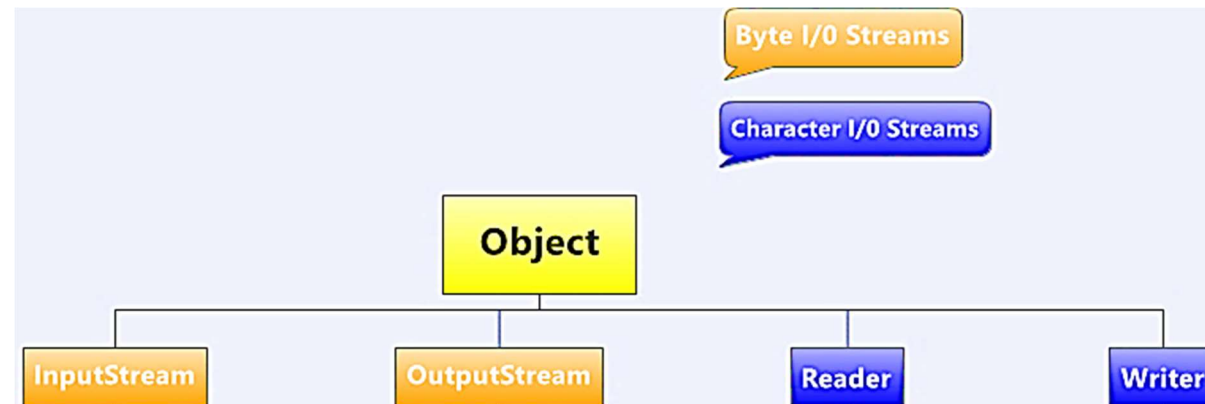
3- Entrées/Sorties

3.3.1 Flux d'entrée/sortie



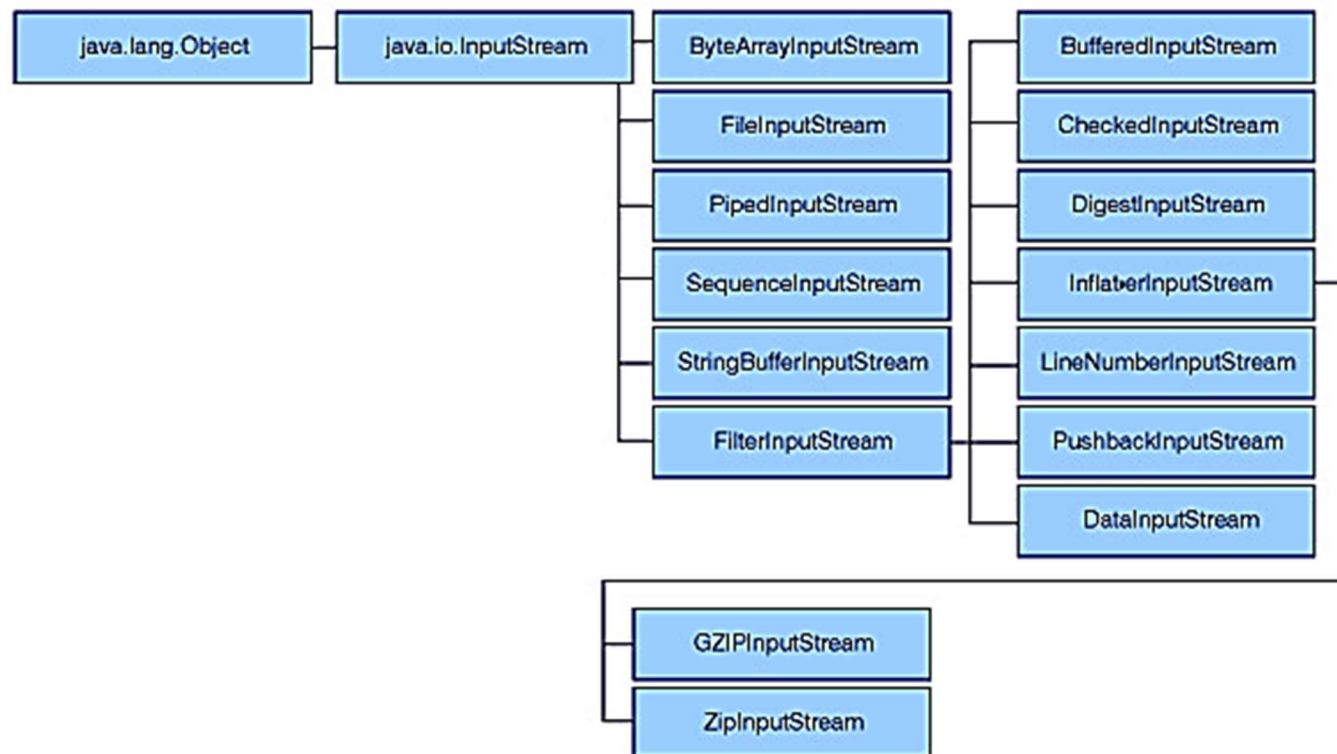
- A chacun de ces quatre types de flux correspond une hiérarchie de classes définie dans le paquetage `java.io`.
- La classe au sommet de cette hiérarchie dans chacun de ces quatre cas est donnée par la table suivante :

	Entrée	Sortie
Octets	InputStream	OutputStream
Caractères	Reader	Writer



3- Entrées/Sorties

3.3.2 Flux d'entrée





3- Entrées/Sorties

3.3.2 Flux d'entrée



- La classe abstraite **InputStream** du package java.io représente un flux d'entrée d'octets.

```
abstract public class InputStream {  
    abstract int read();  
    int read(byte[] b);  
    int readNBytes(byte[] b, int pos, int len);  
    byte[] readNBytes(int len);  
    byte[] readAllBytes();  
    long skip(long n);  
    int available();  
    long transferTo(OutputStream out);  
    void mark(int l);  
    void close();  
    ...  
}
```

- Elle a plusieurs sous-classes qui représentent soit :
 - des **flux d'entrée primaires** (ou bruts), qui obtiennent leurs données d'une source comme un fichier, un tableau d'octets, etc.
 - des **flux d'entrée filtrants**, qui obtiennent leurs données d'un autre flux d'entrée et les transforment ou filtrent à la volée.

3- Entrées/Sorties

3.3.2.1 Méthodes de lecture



- Les différentes méthodes dont le nom commence par **read** permettent de lire un ou plusieurs octets :
 - **int read()** : Lit et retourne le prochain octet sous la forme d'une valeur comprise entre 0 et 255 inclus, ou -1 si la fin du flux a été atteinte.
 - **int read(byte[] b)** : Lit des octets et les place dans le tableau b et retourne le nombre d'octets lus.
 - **int readNBytes(byte[] b, int pos, int len)** : Lit au plus len octets du flux, les place dans le tableau b à partir de la position pos et retourne le nombre d'octets lus ; ce nombre n'est inférieur à len que si la fin du flux a été atteinte durant la lecture.
 - **byte[] readNBytes(int len)** : similaire à la méthode précédente, mais les octets lus sont placés dans un nouveau tableau de bonne taille, qui est retourné.

3- Entrées/Sorties

3.3.2.1 Méthodes de lecture



- **byte[] readAllBytes()** : Lit la totalité des octets restants dans le flux et les place dans un tableau de bonne taille, qui est retourné.
- La méthode **skip** permet d'ignorer des octets :
 - **long skip(long n)** : Ignore (ou saute) au plus n octets du flux et retourne le nombre d'octets ignorés.
- Les méthodes **read** et **skip** sont potentiellement bloquantes. Cela signifie que si la totalité des octets demandé à ces méthodes n'est pas encore disponible mais le sera dans le futur — par exemple : après avoir été lus du disque ou reçus du réseau — le programme est bloqué jusqu'à l'arrivée des octets.
 - Cette caractéristique facilite la programmation mais peut nuire aux performances ou à l'interactivité de l'application. L'un des buts de **java.nio** est d'ailleurs de fournir des méthodes permettant **l'accès non bloquant aux données**.

3- Entrées/Sorties

3.3.2.1 Méthodes de lecture



- **Pour éviter de bloquer le programme, il peut être utile de connaître le nombre d'octets qu'il est possible d'obtenir du flux sans blocage. C'est le but de la méthode **available** :**
 - **int available()** : Retourne une estimation du nombre d'octets disponible qu'il est possible de lire ou d'ignorer sans bloquer le programme.
- **Il est garanti qu'un appel à readNBytes ou skip demandant le nombre d'octets retourné par available ne bloquera pas le programme.**



3- Entrées/Sorties

3.3.2.2 Représentation des octets



● Le type **byte** de Java représente un octet signé dont la valeur est comprise entre -128 et +127. Cela implique une différence importante entre les deux principales variantes de la méthode `read` :

- celle qui retourne le prochain octet en tant qu'entier **int** le retourne sous forme d'une valeur comprise entre 0 et 255 inclus, -1 étant utilisé pour signaler la fin du flux.
- celle qui stocke les octets dans un tableau de type **byte[]** les stocke sous forme de valeurs comprises entre -128 et +127 inclus, -1 étant une valeur normale n'indiquant pas forcément la fin du flux.



3- Entrées/Sorties

3.3.2.2 Représentation des octets



- Par exemple, étant donné un flux **s** contenant les deux octets suivants (donnés ici en binaire) : 00000000 11111111

- Le morceau de programme ci-dessous :

```
System.out.println(s.read() + "," + s.read());
```

- affiche 0,255 tandis que celui ci-dessous

```
byte[] b = new byte[2];
```

```
s.read(b);
```

```
System.out.println(b[0] + "," + b[1]);
```

- affiche 0,-1





- La méthode **transferTo** permet de transférer la totalité des octets restants dans un flux de sortie :
 - **long transferTo(OutputStream out)** : lit tous les octets du flux et les écrit dans le flux de sortie out, puis retourne le nombre d'octets ainsi transféré.
- Les méthodes **de marquage**, peu utilisées, permettent de marquer une position dans le flux et d'y retourner plus tard :
 - **boolean markSupported()** : Retourne vrai si et seulement si le flux gère le marquage.
 - **void mark(int l)** : Place la marque à l'endroit actuel et promet que si la méthode reset est appelée plus tard, elle le sera lorsqu'au plus l octets auront été lus depuis la position actuelle.
 - **void reset()** : Revient à la position marquée précédemment.



3- Entrées/Sorties

3.3.2.4 Méthode de fermeture



- **Souvent, à un flux est associé une ressource du système d'exploitation sous-jacent. Par exemple, l'ouverture d'un fichier provoque l'allocation de mémoire au niveau du système, mémoire qui n'est libérée que lorsque le fichier sera fermé.**
 - Pour cette raison, les flux ont tous une méthode **close** permettant de fermer le flux et de libérer les éventuelles ressources du système qui lui sont associées :
 - **void close()** : Ferme le flux, libérant ainsi les éventuelles ressources associées et rendant par là même le flot inutilisable.
 - **Attention** : Après fermeture, un flux est **inutilisable** et l'appel de n'importe laquelle de ses méthodes — autre que **close** — provoque une exception.

3- Entrées/Sorties

3.3.3 Flux d'entrée d'octets concrets



- Les sous-classes concrètes de **InputStream** sont réparties en deux familles :
 - celles qui représentent des **flux d'entrée primaires**, dont les données proviennent d'une source déterminée (fichier, tableau d'octets, etc.) et qui héritent généralement directement de **InputStream**.
 - celles qui représentent des **flux d'entrée filtrants**, dont les données proviennent d'un flux sous-jacent et qui héritent généralement de **FilterInputStream**, elle-même une sous-classe de **InputStream**.
- Les flots représentant « l'entrée standard » stockés dans l'attribut statique **in** de la classe **System** sont de type **InputStream**.
- Pour manipuler les flux de la console, il est possible de récupérer une instance de **Console** associée à la JVM courante en appelant la méthode **System.console()**.





- La classe **FileInputStream** représente un **flux d'entrée d'octets primaire** dont les octets proviennent d'un fichier.

- Par exemple, un flux fournissant les octets du fichier nommé **file.bin** peut s'obtenir ainsi :

```
InputStream s = new FileInputStream("file.bin");
```

- La classe **ByteArrayInputStream** représente un **flux d'entrée d'octets primaire** dont les octets proviennent d'un tableau d'octets de type **byte[]**.

- Par exemple, un flux fournissant les octets 1, 2 puis 3 peut s'obtenir ainsi :

```
byte[] b = new byte[]{ 1, 2, 3 };
```

```
InputStream s = new ByteArrayInputStream(b);
```

- Un autre constructeur existe pour ne fournir que les octets dont les index sont dans un intervalle donné.

```
ByteArrayInputStream(byte[] b, int pos, int len);
```





3- Entrées/Sorties

3.3.3.1 Flux d'entrée de fichier et Flux d'entrée de tableau



Exemples

```
import java.io.ByteArrayInputStream;
public class TestByteArrayInputStream {
    public static void main(String[] args) throws IOException {
        byte[] tab = "Bonjour CPI2".getBytes();
        InputStream stream = new ByteArrayInputStream(tab);
        int octet;
        while ((octet = stream.read()) != -1) {
            System.out.print((char) octet);
        }
    }
}
```

Résultat d'affichage

```
import java.io.*;
public class TestFileInputStream {
    public static void main(String[] args) throws IOException {
        try (InputStream stream = new FileInputStream("fichier.txt")) {
            byte[] buffer = new byte[1024]; int nbRead;
            while ((nbRead = stream.read(buffer)) != -1) { // ... }
        }
    }
}
```

Dans le deuxième exemple, on utilise la méthode **read** qui prend un tableau d'octets en paramètre. Cela est plus efficace que de lire le fichier octet par octet.



3- Entrées/Sorties

3.3.3.2 Flux à mémoire tampon



- La classe **BufferedInputStream** représente un **flux filtrant** — qui ne filtre rien du tout, en réalité — ajoutant une **mémoire tampon** au flux sous-jacent.

- Cette classe lit les octets du flux sous-jacent par blocs, les stocke dans un tableau, puis les fournit lorsqu'on les lui demande.

- L'utilisation d'une telle mémoire tampon au-dessus d'un flux dont les données sont coûteuses à obtenir une à une — p.ex. lorsque elles proviennent d'un disque lent — peut améliorer les performances. Elle est donc souvent utilisée sur un flux de type **FileInputStream**, ainsi :

```
InputStream s =  
    new BufferedInputStream(new FileInputStream());
```

- De plus, la classe **BufferedInputStream** gère le marquage, donc ses méthodes **mark** et **reset** peuvent donc être utilisées pour marquer une position lors de la lecture et y revenir ultérieurement.



3- Entrées/Sorties

3.3.3.3 Flux décompressant



- La classe **ZipInputStream** — du paquetage **java.util.zip** — décompresse à la volée les données fournies par le flux sous-jacent au moyen de l'algorithme deflate (un algorithme de compression de données sans pertes qui utilise l'algorithme LZ77 et le codage de Huffman).
 - En d'autres termes, le flux sous-jacent de la classe **ZipInputStream** doit recevoir des données compressées au format ZIP, qu'elle décompresse à la volée pour fournir les données décompressées via la méthode **read**.
- La classe **GZIPInputStream** — du paquetage **java.util.zip** — décompresse à la volée les données fournies, qui sont compressées au format gzip, à partir du flux sous-jacent en utilisant l'algorithme deflate.



3- Entrées/Sorties

3.3.3.4 Empilement des flux



● Il est fréquent d'empiler plusieurs flux filtrants sur un flux primaire.

- Par exemple, pour lire un fichier nommé in.zip, en utilisant une mémoire tampon, on peut écrire :

```
InputStream s =  
    new ZipInputStream(  
        new BufferedInputStream(  
            new FileInputStream("in.zip")));
```

- Dans un tel empilement, la fermeture du flux au sommet — ici référencé par s — provoque la fermeture de la totalité des flux qui le composent.



3- Entrées/Sorties

3.3.3.4 Empilement des flux



Exemple :

- L'extrait suivant **compte le nombre d'octets nuls** dans le **fichier in.zip**

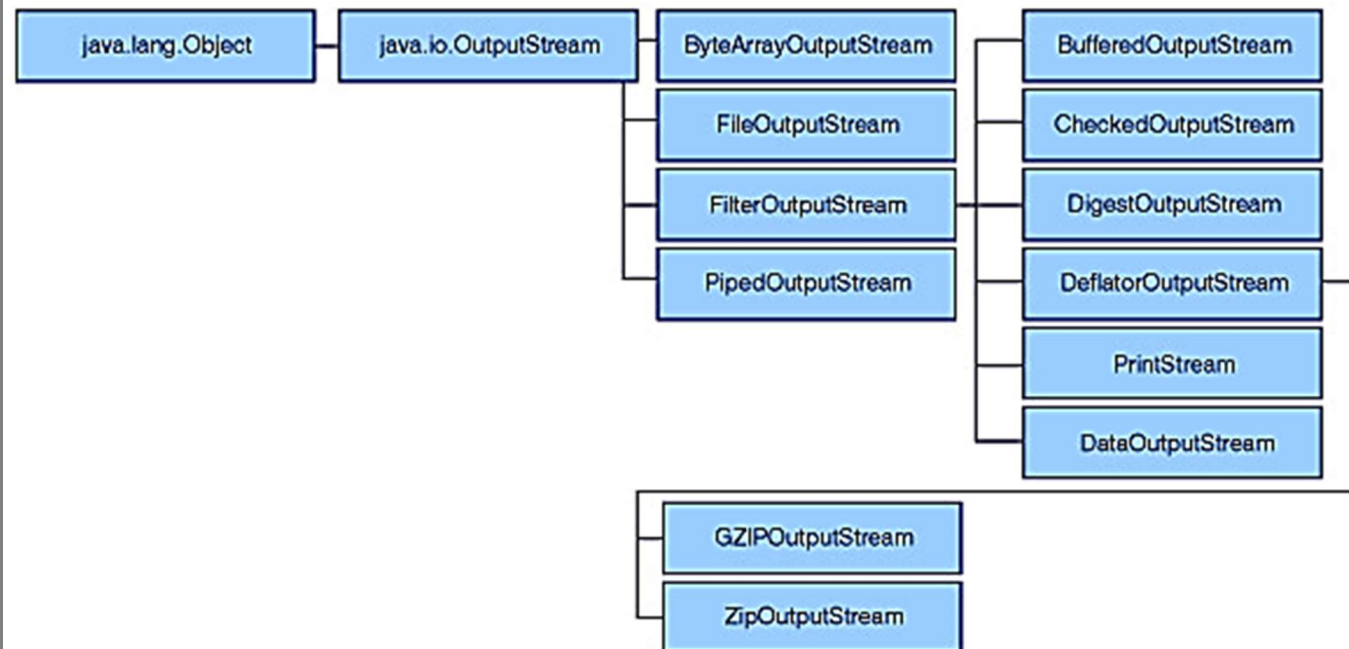
```
InputStream s =  
    new ZipInputStream(  
        new BufferedInputStream(  
            new FileInputStream("in.zip"))));  
  
int b, c = 0;  
while ((b = s.read()) != -1) {  
    if (b == 0)  
        c++;  
}  
s.close();  
System.out.println(c);
```





3- Entrées/Sorties

3.3.4 Flux de sortie d'octets



3- Entrées/Sorties

3.3.4 Flux de sortie d'octets



- La classe abstraite **OutputStream** du package `java.io` représente un flux de sortie d'octets.

```
abstract public class OutputStream {  
    abstract void write(int b);  
    void write(byte[] b);  
    void write(byte[] b, int pos, int len);  
    void flush();  
    void close();  
    ...  
}
```

- Tout comme **InputStream**, elle possède deux grandes catégories de sous-classes, celles représentant des **flux de sortie primaires** — à destination de fichiers par exemple — et celles représentant des **flux de sortie filtrants**, qui envoient les données à un flux sous-jacent en les modifiant éventuellement au passage.



3- Entrées/Sorties

3.3.4.1 Méthodes d'écriture



● Les trois variantes de la méthode **write** permettent d'écrire un ou plusieurs octets dans le flux :

- **void write(int b)** : Ecrit l'octet correspondant dans le flux.
- **void write(byte[] b, int pos, int len)** : Ecrit les len octets obtenus du tableau b à partir de la position pos dans le flux.
- **void write(byte[] b)** équivaut à `write(b, 0, b.length)`.

● Les variantes de la méthode **write** ne garantissent pas que les données sont écrites immédiatement dans le flux.
Pour s'en assurer, on peut utiliser la méthode **flush** :

- **void flush()** : Force les données du flot à être écrites effectivement, par exemple sur le disque ou sur la console.



3- Entrées/Sorties

3.3.4.2 Méthode de fermeture



- Tout comme les flux d'entrée d'octets, les flux de sortie d'octets possèdent une méthode **close** permettant de fermer le flux :
 - **void close()** : Ferme le flux, libérant ainsi les éventuelles ressources associées et rendant par là même le flux inutilisable.



3- Entrées/Sorties

3.3.4.3 Flux de sortie d'octets concrets



- La hiérarchie des sous-classes de **OutputStream** est très similaire à celle des sous-classes de **InputStream**.
- Toutes les classes concrètes de flux d'entrée d'octets décrites précédemment ont un équivalent dans les flux de sortie d'octets.
 - Par exemple, à la classe **FileInputStream** correspond la classe **FileOutputStream**, et ainsi de suite.
- La classe **PrintStream** ajoute à la classe **FilterOutputStream** des méthodes **print**, **println** et **printf** permettant d'écrire des représentations textuelles de données dans le flux.
 - Les flux représentant la « sortie standard » et « l'erreur standard », stockés respectivement dans les attributs statiques **out** et **err** de la classe **System** sont de type **PrintStream**.





3- Entrées/Sorties

3.3.4.3 Flux de sortie d'octets concrets



Exemples

```
import java.io.ByteArrayOutputStream; import java.util.Arrays;
public class TestByteArrayOutputStream {
    public static void main(String[] args) {
        ByteArrayOutputStream st = new ByteArrayOutputStream();
        for (byte b : "Bonjour CPI2".getBytes()) { st.write(b); }
        byte[] byteArray = st.toByteArray();
        System.out.println((char)byteArray[0]);
        System.out.println(Arrays.toString(byteArray));
    }
}
```

Résultat d'affichage

```
import java.io.*;
public class TestFileOutputStream {
    public static void main(String[] args) throws IOException {
        try (FileOutputStream st = new FileOutputStream("fich.txt"))
        { byte[] octets = "Bonjour CP2I".getBytes();
          st.write(octets);
        } } }
```

Dans le deuxième exemple, on utilise la méthode **write** qui prend un tableau d'octets en paramètre. Cela est plus efficace que d'écrire dans le fichier octet par octet.



3- Entrées/Sorties

3.3.5 Ressources



- On l'a vu précédemment, il est important que les flux — entrées et sorties — soient fermés au moyen de la méthode `close` lorsqu'ils sont devenus inutiles.

```
InputStream s = new FileInputStream();
try {
    // ... code utilisant le flux s
} finally {
    s.close();
}
```

- Depuis Java 7, l'instruction « try-with-resources » permet d'écrire simplement un code utilisant une ressource.

```
try (InputStream s = new FileInputStream()) {
    // ... code utilisant le flot s
}
```

- La méthode `close` ne doit plus être appelée explicitement, cela est fait automatiquement à la fin du bloc `try`.

3- Entrées/Sorties

3.3.5 Ressources



- l'instruction « try-with-resources » peut être utilisé avec plusieurs variables ressources. Par exemple :

```
try(InputStream i = new FileInputStream("in.bin");  
    OutputStream o = new FileOutputStream("out.bin"))  
{  
    // ... code utilisant les flots i et o  
} catch (IOException e) {  
    // ... code gérant l'exception  
}
```

- Pour être utilisée dans l' instruction « try-with-resources », une ressource doit implémenter l'interface **AutoCloseable** du package java.lang, définie ainsi :

```
public interface AutoCloseable {  
    void close();  
}
```

- Cette interface est implémentée par toutes les classes de l'API Java possédant une méthode **close**, parmi lesquelles figurent **InputStream**, **OutputStream**, **Reader** et **Writer**.



3- Entrées/Sorties

3.3.6 Entrées/Sorties textuelles



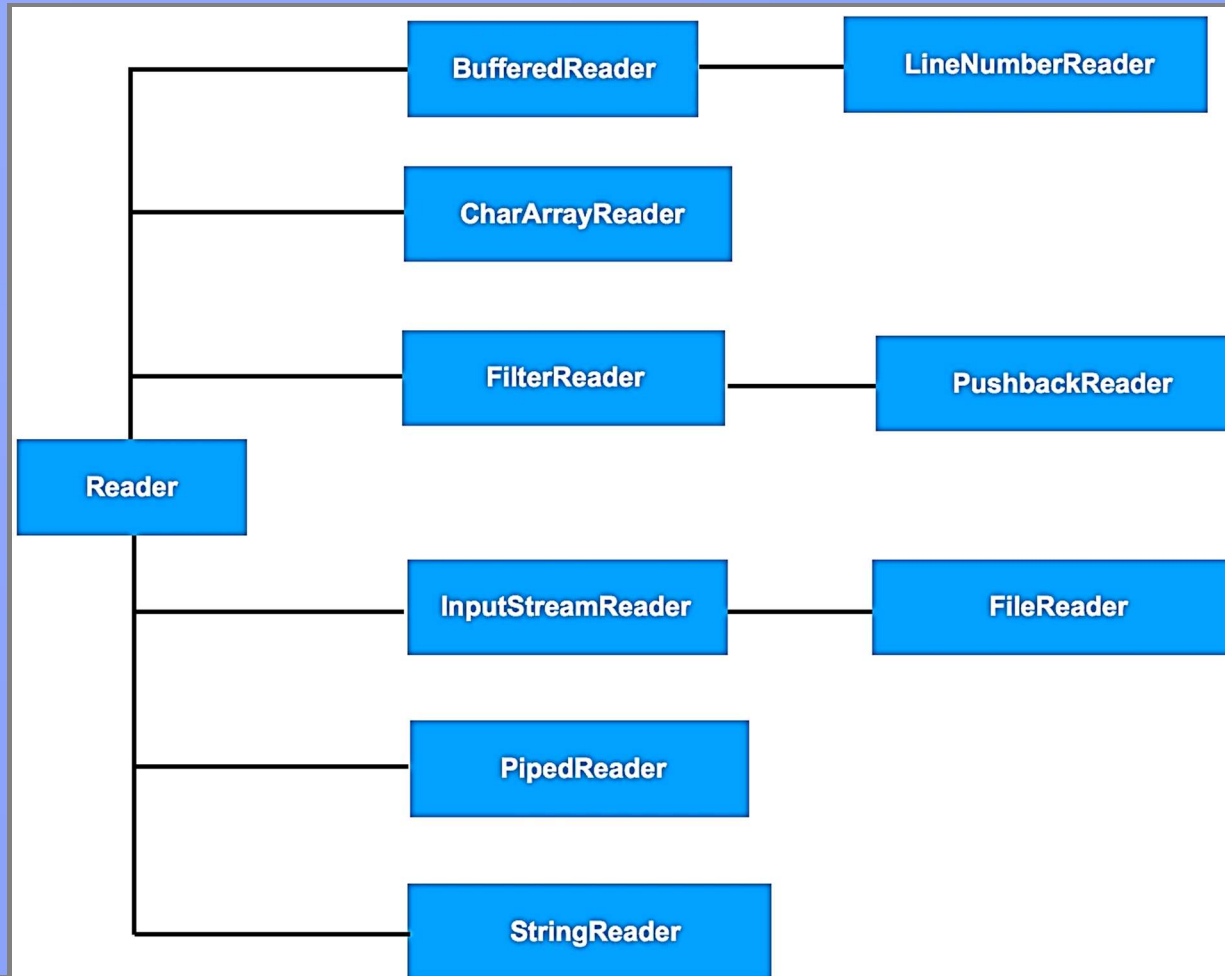
- Un reader (**lecteur**) est un flux d'entrée de caractères ; un writer (**écrivain**) est un flux de sortie de caractères.
 - La classe **Reader** joue le rôle de **InputStream**, tandis que la classe **Writer** joue celui de **OutputStream**.
- Ces deux hiérarchies de classes sont très similaires, et les concepts se retrouvent, avec des noms similaires. Par exemple, **FileReader** est l'équivalent de **FileInputStream**.
- Les **readers** qui doivent détecter les fins de ligne considèrent généralement que n'importe laquelle des trois séquences suivantes termine une ligne :
 - CR (retour de chariot)
 - LF (saut de ligne)
 - CR puis LF
- Les **writers** qui doivent écrire une fin de ligne utilisent la propriété système nommée **System.lineSeparator()** : une chaîne dont la valeur s'obtient par l'appel suivant :
`System.getProperty("line.separator") ;`





3- Entrées/Sorties

3.3.6.1 Readers





- La classe abstraite **Reader** est l'équivalent de `InputStream` mais travaille sur des caractères Java plutôt que des octets :

```
abstract public class Reader {  
    int read();  
    abstract int read(char[] c, int pos, int len);  
    int read(char[] c);  
    long skip(long n);  
    boolean ready();  
    long transferTo(Writer out);  
    void mark(int l);  
    abstract void close();  
    ...  
}
```

- Contrairement à `InputStream`, **Reader** ne possède pas de méthode **`available()`** permettant de connaître une estimation du nombre de caractères pouvant être lus sans bloquer.

3- Entrées/Sorties

3.3.6.1 Readers



- Au lieu de cela, la classe Reader possède la méthode **ready()** qui retourne vrai s'il est certain que le prochain appel à **read()** **sans argument** (qui lit un seul caractère) ne bloquera pas le programme.
 - La raison de cette différence est que fournir une méthode **available()** demanderait de décoder les octets disponibles en caractères, ce qui peut être coûteux.
- **Plusieurs sous-classes de Reader permettent la création de flux pour chaînes de caractères.**
 - Les classes **CharArrayReader**, **StringReader** et **FileReader** permettent de lire des flux de caractères en provenance de tableaux de char, de chaînes de caractères, ou de fichiers.



3- Entrées/Sorties

3.3.6.1 Readers



Exemples

```
import java.io.IOException;
import java.io.Reader;
import java.io.StringReader;
public class TestStringReader {
    public static void main(String[] args) throws IOException {
        Reader reader = new StringReader("Bonjour CPI2");
        int caractere;
        while ((caractere = reader.read()) != -1) {
            System.out.print((char) caractere); }
    }}

```

Résultat d'affichage

```
import java.io.FileReader;
import java.io.IOException;
import java.io.Reader;
public class TestFileReader {
    public static void main(String[] args) throws IOException {
        try (Reader reader = new FileReader("fichier.txt"))
        {   char[] buffer = new char[1024];
            int nbRead;
            while ((nbRead = reader.read(buffer)) != -1) {
                //...
            }
        }}
    }}

```





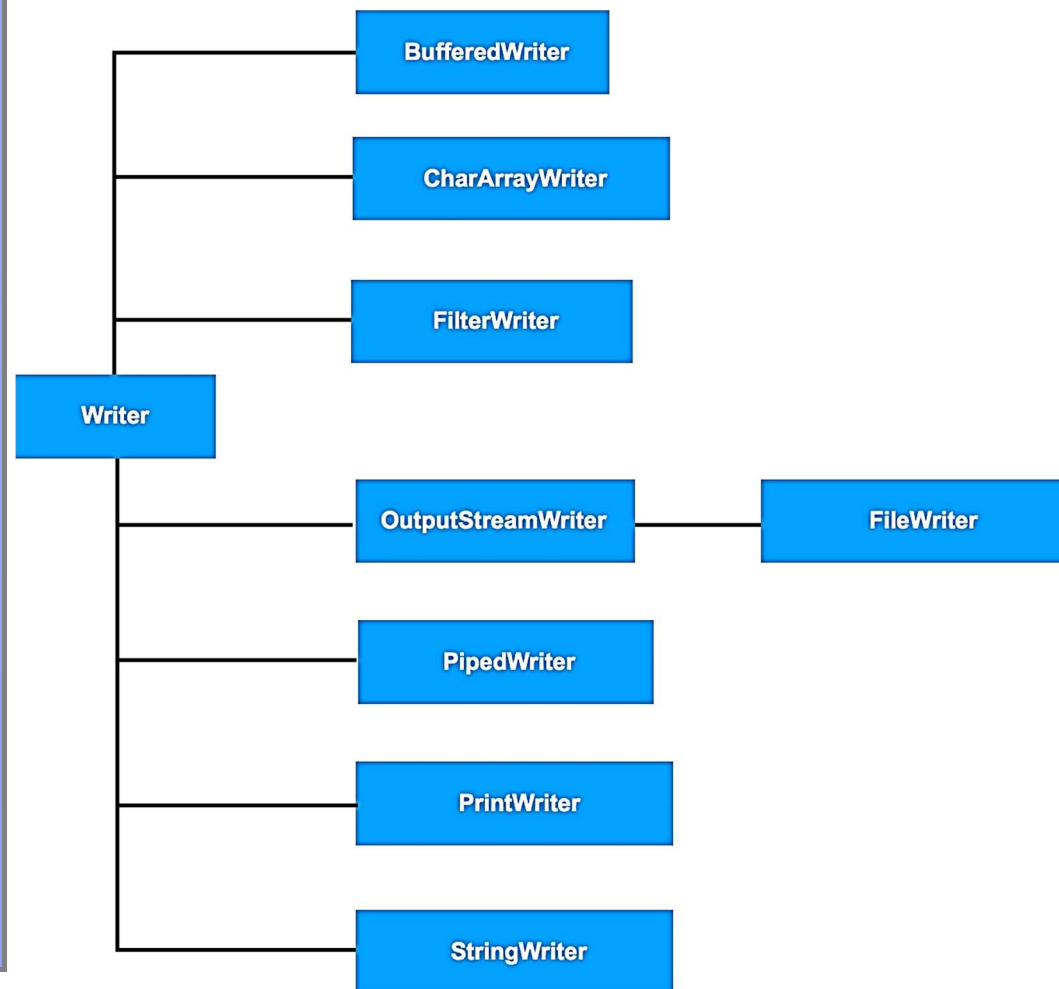
3- Entrées/Sorties

3.3.6.2 Writers

P.O.O : Java avancé



Sofiane HACHICHA
ISIMG 2025 - 2026





- La classe abstraite **Writer** est l'équivalent de **OutputStream** qui permet d'écrire des flux de caractères plutôt que des octets :

```
abstract public class Writer {  
    void write(int b);  
    void write(char[] b);  
    abstract void write(char[] b, int pos, int len);  
    void write(String str);  
    void write(String str, int pos, int len);  
    Writer append(char c);  
    Writer append(CharSequence c);  
    Writer append(CharSequence c, int start, int end);  
    abstract void flush();  
    abstract void close();  
    ...  
}
```

- La classe **Writer** fournit des méthodes **write** qui acceptent en paramètre des caractères.





- La classe **Writer** fournit également des méthodes d'écriture **append** équivalentes aux méthodes **write** et qui retournent une instance de **Writer** afin de pouvoir chaîner les appels.
- Le lien entre les readers/writers et les flux d'octets est fait par deux classes :
 - **InputStreamReader** qui, étant donné un flux d'entrée d'octets et un encodage de caractères, fournit un reader.
 - **InputStreamReader** hérite de **Reader** et prend comme paramètre de constructeur une instance de **InputStream**.
 - **OutputStreamWriter** qui, étant donné un flux de sortie d'octets et un encodage de caractères, fournit un writer.
 - **OutputStreamWriter** hérite de **Writer** et prend comme paramètre de constructeur une instance de **OutputStream**.



3- Entrées/Sorties

3.3.6.3 Readers/Writers



Exemple

```
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.FileInputStream;
public class TestInputStreamReader {
public static void main(String[] args) throws IOException {
try(InputStreamReader r =
    new InputStreamReader(new FileInputStream("f.txt"), "UTF-8")) {
    int nbRead;
    while ((nbRead = r.read()) != -1) {
        System.out.print((char)nbRead);    }
    }
}
```



3- Entrées/Sorties

3.3.6.3 Readers/Writers



- La classe **FileReader**, qui hérite de la classe **InputStreamReader**, représente un reader dont les caractères proviennent d'un fichier.
 - Son constructeur prend en argument le nom du fichier ainsi que l'encodage à utiliser pour transformer les octets du fichier en caractères.
- La classe **FileWriter**, qui hérite de la classe **OutputStreamWriter**, représente un writer dont les caractères sont écrits dans un fichier.
 - Son constructeur prend les mêmes arguments que le constructeur de **FileReader**.
 - Son constructeur peut prendre un booléen en second paramètre qui permet de spécifier si le fichier doit être ouvert en ajout (append). S'il est vrai, le flux de sortie ne rafraîchit pas le fichier pointé (les caractères seront écrits à la fin du fichier plutôt qu'au début).



3- Entrées/Sorties

3.3.6.3 Readers/Writers



- La classe **CharArrayReader** représente un reader dont les caractères proviennent d'un tableau de caractères de type `char[]` passé au constructeur.
- La classe **CharArrayWriter** représente un writer écrivant ses caractères dans un tableau de caractères de type `char[]`. Celui-ci peut être obtenu au moyen de la méthode **toCharArray**.
- Les classes **StringReader** et **StringWriter** sont similaires mais basées sur des chaînes de caractères Java de type `String` plutôt que sur des tableaux.



3- Entrées/Sorties

3.3.6.3 Readers/Writers



Exemples

```
import java.io.StringWriter;
public class TestStringWriter {
    public static void main(String[] args){
        StringWriter writer = new StringWriter();
        writer.append("Bonjour")
            .append(' ')
            .append("CPI2");
        System.out.println(writer.toString());
    }
}
```

Résultat d'affichage

```
import java.io.*;
public class TestFileWriter {
    public static void main(String[] args) throws IOException {
        try (FileWriter writer = new FileWriter("fich.txt", true))
        { writer.append("Bonjour CPI2!\n"); }
    }
}
```

Dans le deuxième exemple, le constructeur **FileWriter** ne permet pas de positionner l'encodage de caractères (*charset*) utilisé pour écrire dans le fichier. On utilise l'encodage par défaut de la JVM qui est dépendant du système.



3- Entrées/Sorties

3.3.6.3 Readers/Writers



- A l'image de **BufferedInputStream** et **BufferedOutputStream**, les classes **BufferedReader** et **BufferedWriter** ajoutent une mémoire tampon au reader (respectivement writer) sous-jacent, améliorant ainsi les performances du programme.
- **BufferedReader** ajoute une méthode au reader :
 - **String readLine()** : Lit et retourne la prochaine ligne du reader sous-jacent, ou null si la fin a été atteinte,
- **BufferedWriter** ajoute une méthode au writer :
 - **void newLine()** : Ecrit une fin de ligne dans le writer sous-jacent. Elle est utilisée pour séparer la ligne suivante en une nouvelle ligne.

3- Entrées/Sorties

3.3.6.3 Readers/Writers



● **LineNumberReader** est une sous-classe de **BufferedReader** qui garde trace du numéro de ligne actuel — en commençant à 0 — et permet de le consulter ou modifier à l'aide des méthodes suivantes :

- **int getLineNumber()** : Retourne le numéro de ligne actuel
- **void setLineNumber(int l)** : Définit le numéro de ligne à la valeur donnée sans que cela n'affecte le reader sous-jacent de quelque manière que ce soit.



3- Entrées/Sorties

3.3.6.3 Readers/Writers



Exemples

Dans cet exemple, on crée un `BufferedWriter` ayant une mémoire tampon de 1 Ko.

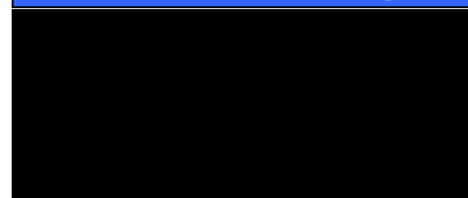
```
import java.io.*;

public class TestBufferedWriter {
    public static void main(String[] args) throws IOException {
        try (BufferedWriter w =
            new BufferedWriter(new FileWriter("f.txt", true), 1024))
        { w.append("Bonjour"); w.newLine(); w.append("Au revoir"); }
    }
}
```

```
import java.io.*;

public class TestLineNumberReader {
    public static void main(String[] args) throws IOException {
        LineNumberReader r =
            new LineNumberReader(new StringReader("abc\nd"));
        String l;
        while ((l = r.readLine()) != null)
            System.out.println(l);
        System.out.println("Nombre de lignes lues: " + r.getLineNumber());
    }
}
```

Résultat d'affichage



3- Entrées/Sorties

3.3.6.3 Readers/Writers



- L'extrait du programme, ci-dessous, lit un fichier nommé **a1.txt** encodé en UTF-8 et l'écrit en UTF-16 dans **a2.txt** :

```
try (Reader i = new FileReader("a1.txt", StandardCharsets.UTF_8);  
    Writer o = new FileWriter("a2.txt", StandardCharsets.UTF_16)) {  
    i.transferTo(o);  
}
```

- Dans cet exemple, il est nécessaire d'importer la classe **java.nio.charset.StandardCharsets**
- Le paquetage **java.nio.charset** contient un ensemble de classes permettant de travailler avec des encodages de caractères.
 - la classe **Charset** représente un encodage de caractères, et celui utilisé par défaut peut être obtenu au moyen de la méthode statique **defaultCharset**.
 - la classe **StandardCharsets** accède, via des attributs statiques, aux encodages les plus utilisés : ASCII, ISO 8859-1, UTF-8 et les trois variantes de UTF-16.



3- Entrées/Sorties

3.4 Fichiers et chemins



- Le paquetage **java.io** contient également la classe **File** qui représente un fichier. À travers cette classe, il est possible de savoir si le fichier existe, s'il s'agit d'un répertoire, etc.
 - On peut également créer le fichier ou le supprimer.
- Pour représenter le chemin d'accès à un fichier, on peut utiliser une chaîne de caractère représentant ce chemin.
 - L'inconvénient de cette méthode est qu'elle n'est pas portable suivant les différents systèmes de fichiers et les différents systèmes d'exploitation.
 - Pour résoudre ce problème en utilisant le paquetage **java.nio.file**, on doit tirer profit de l'interface **Path** qui permet de représenter un chemin de fichier et exploiter les classes **Paths** et **FileSystem** qui servent à construire des instances de type **Path**. La classe **FileSystem** fournit des méthodes pour obtenir des informations sur le système de fichiers présent sur la machine. On accède à une instance de **FileSystem** par la méthode **FileSystem.getDefault()**. 181





3- Entrées/Sorties

3.4 Fichiers et chemins



Exemple

```
import java.io.File;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
public class TestFile {
    public static void main(String[] args) throws IOException {
        File f = new File("fich.txt");
        if (!f.exists()) { f.createNewFile(); }
        if (f.canWrite()) {
            try (BufferedWriter w = new BufferedWriter(new FileWriter(f)))
            { w.write("Bonjour"); }
        }
        f.delete();
    }
}
```



3- Entrées/Sorties

3.4 Fichiers et chemins

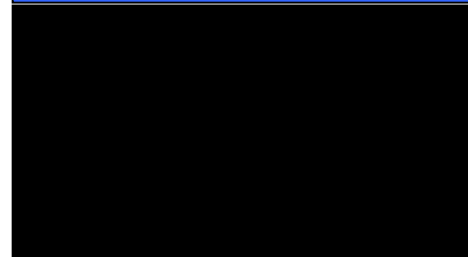


Exemple

```
import java.io.*;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.FileSystems;

public class TestPath{
    public static void main(String[]args) throws IOException{
        Path ch = Paths.get("E:", "Doc", "fich.txt");
        System.out.println(ch);
        System.out.println(ch.getNameCount());
        System.out.println(ch.getParent());
        System.out.println(ch.getFileName());
        ch=FileSystems.getDefault().getPath("E:", "Doc", "fich.txt");
        File f = ch.toFile();
        if (f.canWrite()) {
            try (BufferedWriter w = new BufferedWriter(new FileWriter(f)))
            { w.write("Au revoir!"); }
        }
    }
}
```

Résultat d'affichage



3- Entrées/Sorties

3.4 Fichiers et chemins



- Pour les opérations les plus courantes sur les fichiers, la classe **java.nio.file.Files** fournit un ensemble de méthodes statiques qui permettent de créer, de consulter, de modifier ou de supprimer des fichiers et des répertoires en utilisant un minimum d'appel.
- Cette classe se révèle très pratique d'utilisation notamment pour lire l'intégralité d'un fichier.
- L'exemple, figurant dans la diapositive 180, peut s'écrire d'une manière plus pratique :

```
Path source = Paths.get("a1.txt");  
Path cible = Paths.get("a2.txt");  
try (BufferedReader i =  
    Files.newBufferedReader(source, StandardCharsets.UTF_8);  
    BufferedWriter o =  
    Files.newBufferedWriter(cible, StandardCharsets.UTF_16)) {  
    i.transferTo(o);  
}
```



3- Entrées/Sorties

3.4 Fichiers et chemins



Exemple

```
import java.io.*; import java.util.List;
import java.nio.file.Files;
import java.nio.file.Path; import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
public class TestFiles {
    public static void main(String[] args) throws IOException {
        Path f1 = Paths.get("fich1.txt");
        // création du fichier
        f1 = Files.createFile(f1);
        System.out.println("Taille du fichier : " + Files.size(f1));
        try (BufferedWriter writer = Files.newBufferedWriter(f1,
            StandardOpenOption.WRITE))
        { writer.append("Hello !\n").append("Bye !\n"); }
        System.out.println("Taille du fichier : " + Files.size(f1));
        // Copie vers un nouveau fichier
        Path f2 = Paths.get("fich2.txt");
        Files.copy(f1, f2);
        // Lecture de l'intégralité du fichier
        List<String> l = Files.readAllLines(f1);
        // Suppression du fichier1
        Files.deleteIfExists(f1);
        System.out.println(l);
    }
}
```

Résultat d'affichage



3- Entrées/Sorties

3.5 La sérialisation d'objets



- La sérialisation c'est transformer un objet (en mémoire) en une suite d'octets le représentant.

objet $\xrightarrow{\text{sérialisation}}$ suite de bytes $\xrightarrow{\text{désérialisation}}$ objet

objet o ds JVM₁ $\longrightarrow$ suite de bytes $\longrightarrow \left(\begin{array}{c} \text{fichier} \\ \text{réseau} \end{array} \longrightarrow \right)$ objet o ds JVM₂

- Les classes **ObjectOutputStream** et **ObjectInputStream** héritent respectivement de **OutputStream** et **InputStream**. Elles permettent la sérialisation et la désérialisation d'objets en Java :

- Un objet, ainsi que tous les objets qu'il référence, peuvent être écrits dans un flux ou lus à partir d'un flux.

- Cette fonctionnalité est utile pour sauvegarder l'état d'une application dans un fichier ou bien pour échanger des données entre deux programmes Java via un réseau.

3- Entrées/Sorties

3.5 La sérialisation d'objets



● La sérialisation d'objets a des limites :

- Seul l'état des objets est écrit ou lu, cela signifie que les fichiers class ne font pas partie de la sérialisation et doivent être disponibles pour la JVM au moment de la lecture (opération de désérialisation réalisée avec la classe `ObjectInputStream`).
- Le format des données sérialisées est propre à Java, ce mécanisme n'est donc pas adapté pour échanger des informations avec des applications qui ne seraient pas écrites en Java.
- Les données sérialisées sont très dépendantes de la structure des classes. Si des modifications sont apportées à ces dernières, une grappe d'objets préalablement sérialisée dans un fichier ne sera sans doute plus lisible.



3- Entrées/Sorties

3.5 La sérialisation d'objets

- Pour qu'un objet puisse être sérialisé, sa classe doit implémenter **l'interface marqueur Serializable**, qui est une interface vide sans aucun champ ni méthode. De plus, si cet objet référence d'autres objets dans ses attributs, alors les classes de ces objets référencés doivent également implémenter l'interface Serializable.
- Il est important de noter que de nombreuses classes de l'API standard de Java implémentent déjà l'interface Serializable, y compris des classes couramment utilisées telles que String.
- Si on tente de sérialiser un objet dont la classe n'implémente pas Serializable, cela entraînera une exception de type `java.io.NotSerializableException`.

3- Entrées/Sorties

3.5 La sérialisation d'objets



- Parfois, il est nécessaire d'exclure certaines informations d'une classe de la sérialisation. Cela peut résulter de limitations techniques ou de la sensibilité des données qui ne doivent pas être incluses dans la sérialisation.
 - Pour que les processus de sérialisation/désérialisation ignorent ces attributs lors de l'opération de sérialisation, il faut leur ajouter le mot-clé **transient**.
- Prenons comme exemple la classe Game suivante :

```
public class Game implements java.io.Serializable {  
    private String nom, style;  
    private double prix;  
    public Game(String nom, String style, double prix) {  
        this.nom = nom;  
        this.style = style;  
        this.prix = prix;  
    }  
    public String toString() {  
        return "Nom du jeu : " + this.nom + "\nStyle de jeu : " +  
            this.style + "\nPrix du jeu : " + this.prix + "\n";  
    }  
}
```

3- Entrées/Sorties

3.5 La sérialisation d'objets



● Le code suivant sérialise les données dans le fichier game.bin

```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Paths;

public class TestSerialisation {
    public static void main(String[] args) throws IOException {
        try(OutputStream os = Files.newOutputStream(Paths.get("game.bin"))) {
            ObjectOutputStream oos = new ObjectOutputStream(os) {
                oos.writeObject(new Game("Assassin Creed", "Aventure", 45.69));
                oos.writeObject(new Game("Tomb Raider", "Plateforme", 23.45));
                oos.writeObject(new Game("Tetris", "Stratégie", 2.50));
            }
        }
    }
}
```

3- Entrées/Sorties

3.5 La sérialisation d'objets



● Le code suivant lit le fichier game.bin pour retrouver les objets Game.

```
import java.io.IOException;
import java.io.InputStream;
import java.io.ObjectInputStream;
import java.nio.file.Files;
import java.nio.file.Paths;

public class TestDeserialisation {
    public static void main(String[] args) throws IOException,
        ClassNotFoundException {
        try (InputStream is = Files.newInputStream(Paths.get("game.bin")));
            ObjectInputStream ois = new ObjectInputStream(is)) {
            System.out.println("Affichage des jeux :");
            System.out.println("*****");
            System.out.println(((Game)ois.readObject()).toString());
            System.out.println(((Game)ois.readObject()).toString());
            System.out.println(((Game)ois.readObject()).toString());
        }
    }
}
```





Institut Supérieur d'Informatique
et de Multimédia de Gabes

3- Entrées/Sorties

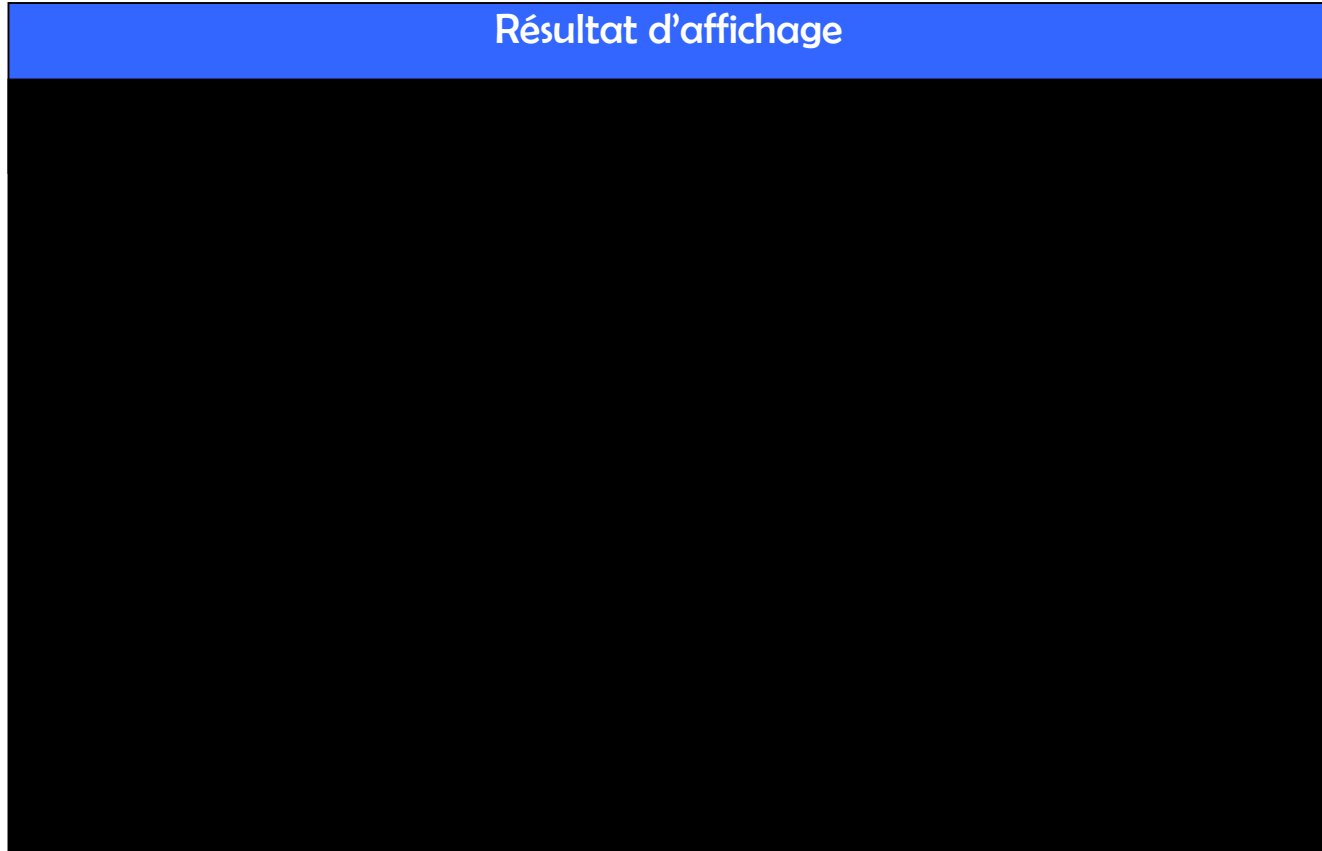
3.5 La sérialisation d'objets

P00 : Java avancé



Sofiane HACHICHA
ISIMG 2025 - 2026

Résultat d'affichage



3- Entrées/Sorties

3.5 La sérialisation d'objets



- Pour exclure l'attribut prix de la sérialisation/désérialisation dans la classe Game afin que le prix de chaque jeu stocké dans le fichier game.bin devienne 0.0, il faut utiliser le mot-clé transient comme suit :

```
public class Game implements java.io.Serializable {  
    private String nom, style;  
    private transient double prix;  
    public Game(String nom, String style, double prix) {  
        this.nom = nom;  
        this.style = style;  
        this.prix = prix;  
    }  
    public String toString() {  
        return "Nom du jeu : " + this.nom + "\nStyle de jeu : " +  
            this.style + "\nPrix du jeu : " + this.prix + "\n";  
    }  
}
```



3- Entrées/Sorties

3.6 serialVersionUID



- **Le serialVersionUID est un identifiant unique de type long utilisé dans les classes Java qui implémentent l'interface Serializable.**
 - Il sert à garantir la compatibilité entre les versions d'une classe lors de la sérialisation et de la désérialisation. Il vérifie que la classe n'a pas trop changé entre la sauvegarde et la lecture d'un objet.
- **Si une classe sérialisée est modifiée (par exemple, en ajoutant ou supprimant des champs) et qu'aucun serialVersionUID explicite n'est déclaré, la JVM génère automatiquement un identifiant basé sur la structure de la classe.**
 - Cela peut entraîner une erreur **InvalidClassException** si un objet sérialisé avec une version précédente de la classe est désérialisé avec une version modifiée.



3- Entrées/Sorties

3.6 serialVersionUID



- En déclarant explicitement un champ serialVersionUID (exemple, **private static final long serialVersionUID = 1L;**), le développeur contrôle la version de la classe et évite ces problèmes de compatibilité. Cette pratique améliore également les performances en évitant que la JVM ne calcule dynamiquement cet identifiant.
- Il est recommandé de définir un serialVersionUID constant si la structure de la classe reste stable et de le mettre à jour uniquement en cas de modifications incompatibles. Des outils comme l'utilitaire serialver du JDK ou les fonctionnalités des IDE (comme Eclipse ou IntelliJ IDEA) permettent de générer automatiquement une valeur pour ce champ.
- Le champ serialVersionUID doit être défini dans toutes les classes qui implémentent directement l'interface Serializable. Même si une classe n'implémente pas directement cette interface mais hérite d'une classe qui l'implémente, il est conseillé de définir un serialVersionUID explicite si elle est susceptible d'être sérialisée.

